Reviewer Pack — Quiver Mutation Neighborhood Lower-Bounds ACC0 by Sensitivit…

Entry c7f898dc68a4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 17:51:31 UTC

Quiver Mutation Neighborhood Lower-Bounds ACC0 by Sensitivity

Entry ID: `c7f898dc68a4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 17:51:31 UTC

1. Conjecture

Field A (mathematical branch): Cluster algebra theory of Fomin–Zelevinsky — finite-step quiver mutation neighborhoods (FZ 2002 'Cluster algebras I'; Keller's mutation-class machinery). Mutations are non-ring combinatorial graph rewrites (arrow reversal at a vertex, path completion, 2-cycle cancellation) on directed multi-graphs; an arXiv search 'quiver mutation' AND ('ACC^0' OR 'circuit complexity' OR 'Boolean circuit') returns no direct hits, with <5 adjacent papers using cluster algebras for combinatorial gadgets — never for circuit lower bounds. **Field B** (complexity object): ACC^0[m] circuit size and depth for Boolean functions, in the Williams 2011 / Murray–Williams 2018 algorithmic-method regime targeting NEXP ⊄ ACC^0 and MOD_q lower bounds for q coprime to m. The invariant is a STRUCTURAL property of the circuit DAG (not of its truth table), aligning with Williams-style structural exploitations of small-circuit structure.

Statement:

> For an ACC^0[m] circuit C with N gates and depth d computing f:{0,1}^n→{0,1} with average sensitivity I(f)=E_x[#{i:f(x⊕e_i)≠f(x)}], let Q(C) be the multi-quiver whose vertices are circuit nodes (inputs and gates) and whose arc multiplicity i→g equals the number of times node i feeds gate g, with 2-cycles cancelled. Define NbhdSize(C):=#{isomorphism classes of μ_v(Q(C)) : v∈V(Q(C))}, where μ_v is the Fomin–Zelevinsky mutation at v (reverse all arrows at v; for each path i→v→j add an arrow i→j with multiplicity mult(i→v)·mult(v→j); cancel resulting 2-cycles), and isomorphism is detected by Weisfeiler–Lehman color refinement on the labeled multi-digraph. Conjecture: NbhdSize(C) ≥ ⌈I(f)/(4(d+1))⌉ for every ACC^0[m] circuit; a single C with NbhdSize(C) < ⌈I(f)/(4(d+1))⌉ refutes the conjecture.

Rationale (proposer's reasoning):

> Quiver mutation is a non-commutative, non-ring graph rewrite that cancels 2-cycles — an operation that does not lift coherently to polynomial extensions of the Boolean ring, placing the invariant outside the Aaronson–Wigderson algebrization frame that has tripped past GCT-flavoured ACC^0 attempts. NbhdSize is a structural circuit invariant rather than a truth-table predicate, so the Razborov–Rudich largeness/usefulness clauses do not apply and the natural-proofs barrier is sidestepped (the same f admits circuits with very different NbhdSize). Heuristically, high-sensitivity targets force the gate-multi-quiver to be mutation-rigid (many non-isomorphic 1-step neighbours), because low-rigidity quivers

correspond to symmetric, low-sensitivity computation — giving a Williams-style structural-versus-functional tension.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e0ef6d199d00d184`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 90 ACC^0[2] trials (n∈{4,5,6}, d∈{2,3}, N∈[n,3n], 30 seeds per cell), compute NbhdSize(C) via WL-3 on μ_v(Q(C)) and I(f) via full truth table. Conjecture is SUPPORTED iff NbhdSize(C) ≥ ⌈I(f)/(4(d+1))⌉ holds in ≥98% of trials with zero strict violations exceeding gap 1; FALSIFIED iff any single trial yields NbhdSize(C) < ⌈I(f)/(4(d+1))⌉.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.80	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- quiver mutation circuit complexity ACC0 lower bound` - `Fomin Zelevinsky mutation Boolean circuit sensitivity` - `Weisfeiler Lehman quiver mutation neighborhood circuit lower bound`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import sys
import random
import math
import itertools
from collections import defaultdict, deque

def generate_circuit(n, d, N, seed):
    random.seed(seed)
    gate_types = ['AND', 'OR', 'MOD-2']
    layers = [[] for _ in range(d + 1)]
    inputs = [f'x{i}' for i in range(n)]
    layers[0] = inputs.copy()
```

```
    for depth in range(1, d + 1):
        for _ in range(N // d):
            if not layers[depth - 1]:
                break
            gate_type = random.choice(gate_types)
            input1 = random.choice(layers[depth - 1])
            input2 = random.choice(layers[depth - 1])
            gate_name = f'{gate_type}_{depth}_{_}'
            layers[depth].append(gate_name)

    circuit = {'inputs': inputs, 'layers': layers, 'output': layers[-1][0] if layers[-1] else None
    return circuit

def build_quiver(circuit):
    quiver = defaultdict(lambda: defaultdict(int))
    layers = circuit['layers']

    for depth in range(1, len(layers)):
        for gate in layers[depth]:
            gate_type, _, _ = gate.split('_')
            for input_gate in layers[depth - 1]:
                quiver[input_gate][gate] += 1

    return quiver

def mutate_quiver(quiver, vertex):
    new_quiver = defaultdict(lambda: defaultdict(int))

    for u in quiver:
        for v in quiver[u]:
            if u == vertex or v == vertex:
                continue
            new_quiver[u][v] = quiver[u][v]

    for u in quiver:
        for v in quiver[u]:
            if u == vertex:
                for w in quiver[v]:
                    if w != vertex:
                        new_quiver[w][v] += quiver[u][v] * quiver[v][w]

    for u in new_quiver:
        for v in new_quiver[u]:
            if u == v:
                del new_quiver[u][v]

    return new_quiver

def weisfeiler_lehman(quiver, max_rounds=3):
    color = {}
    for u in quiver:
        color[u] = u.split('_')[0] if '_' in u else u

    for _ in range(max_rounds):
        new_color = {}
```

```
        for u in quiver:
            neighbors = sorted([(v, quiver[u][v]) for v in quiver[u]], key=lambda x: x[0])
            color_str = f"{color[u]}_{tuple(neighbors)}"
            new_color[u] = color_str
        color = new_color

    return frozenset(color.values())

def compute_sensitivity(circuit, n):
    inputs = circuit['inputs']
    layers = circuit['layers']
    output_gate = circuit['output']

    if not output_gate:
        return 0.0

    truth_table = {}
    for x in itertools.product([0, 1], repeat=n):
        x = list(x)
        values = {g: 0 for layer in layers for g in layer}
        for i, val in enumerate(x):
            values[inputs[i]] = val

        for depth in range(1, len(layers)):
            for gate in layers[depth]:
                gate_type, _, _ = gate.split('_')
                input1 = layers[depth - 1][0] if layers[depth - 1] else None
                input2 = layers[depth - 1][1] if len(layers[depth - 1]) > 1 else None
                if input1 and input2:
                    if gate_type == 'AND':
                        values[gate] = values[input1] and values[input2]
                    elif gate_type == 'OR':
                        values[gate] = values[input1] or values[input2]
                    elif gate_type == 'MOD-2':
                        values[gate] = values[input1] ^ values[input2]

        truth_table[tuple(x)] = values[output_gate]

    sensitivity = 0.0
    for x in truth_table:
        for i in range(n):
            x_flipped = list(x)
            x_flipped[i] = 1 - x_flipped[i]
            sensitivity += truth_table[x] != truth_table[tuple(x_flipped)]

    return sensitivity / (2 ** n)

def run_trial(seed):
    random.seed(seed)
    n = random.choice([4, 5, 6])
    d = random.choice([2, 3])
    N = random.randint(n, min(3 * n, 25))

    circuit = generate_circuit(n, d, N, seed)
    quiver = build_quiver(circuit)
```

```
    mutation_neighborhood = set()
    for vertex in quiver:
        mutated_quiver = mutate_quiver(quiver, vertex)
        signature = weisfeiler_lehman(mutated_quiver)
        mutation_neighborhood.add(signature)

    nbhd_size = len(mutation_neighborhood)
    sensitivity = compute_sensitivity(circuit, n)
    bound = math.ceil(sensitivity / (4 * (d + 1)))

    conjecture_holds = nbhd_size >= bound
    counterexample = f"n={n}, d={d}, N={N}, seed={seed}, nbhd_size={nbhd_size}, sensitivity={sensi

    return {
        "metric_name": "NbhdSize",
        "metric_value": nbhd_size,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        failing_trials = [r for r in results if not r["conjecture_holds"]]
        first_failing_seed = failing_trials[0]["seed"]
        counterexample = failing_trials[0]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6aabdc6d.py", line 160, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6aabdc6d.py", line 136, in run_trial
    mutated_quiver = mutate_quiver(quiver, vertex)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6aabdc6d.py", line 57, in mutate_quiver
    for u in quiver:
RuntimeError: dictionary changed size during iteration
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a RuntimeError (dictionary changed size during iteration) in mutate_quiver before any trials produced data, so neither the support nor falsification condition can be evaluated. | next: Fix mutate_quiver to iterate over a snapshot (e.g., list(quiver.items())) or build the mutated quiver in a fresh dict, then rerun the 90-trial pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	219880
2	preregistration	claude_max	opus	0	0	7288
3	novelty	claude_max	opus	0	0	3103
4	novelty	claude_max	opus	0	0	6208
5	test_gen	mistral	codestral-latest	0	0	317274
6	test_gen	mistral	codestral-latest	0	0	314851
7	test_gen	mistral	codestral-latest	0	0	324130
8	test_gen	mistral	codestral-latest	0	0	322226
9	judge	claude_max	opus	0	0	4075

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1519035 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/c7f898dc68a4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c7f898dc68a4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c7f898dc68a4.tar.gz` (if generated)